

অপারেটিং সিস্টেমের ভূমিকা
(Introduction to Operating Systems)
প্রফেসর চেস্টার রেবেরো
(Prof. Chester Rebeiro)
কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ
(Department of Computer Science and Engineering)
ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাস
(Indian Institute of Technology, Madras)
সপ্তাহ – 06
বক্তৃতা – 25
লকিং এর পদ্ধতি
(How to Implement Locking)
(Software Solutions)

হ্যালো আগের ভিডিওতে আমরা critical section এর একটি ভূমিকা দেখেছি। আমরা দেখেছি যে critical section সমস্যার সমাধানের জন্য 3 টি প্রয়োজনীয়তা রয়েছে। প্রথমটি mutual exclusion, দ্বিতীয়টি progress এবং তৃতীয়টি bounded wait। এই ভিডিওতে, আমরা critical section সমস্যার জন্য কিছু সফ্টওয়্যার সমাধান দেখব।

(স্লাইড সময় 00:43 দেখুন)

আসুন আমরা আরও সহজ সমাধান দিয়ে শুরু করি যা interrupt নিষ্ক্রিয় (disable) করে দেয়, তাই এই সমাধান শুধুমাত্র single core সিস্টেমের জন্য প্রযোজ্য। Multicore system এর সাথে, মূলত সিস্টেমে উন্নত programmable interrupt সিস্টেমে উপস্থিতির কারণে interrupt গুলি বেশ কয়েকটি প্রসেসরের কাছে পাঠানো হয়। অতএব এই সমাধান multicore সিস্টেমের উপর কাজ করবে না। অন্যদিকে, single core প্রসেসর বা single core সিস্টেমের জন্য, interrupt disable কার্যকরী হয়। মূলত আমরা দেখেছি যে যদি interrupt disable

করা হয় তবে context switching আটকান যাবে এবং context switching আটকান গেলে execution চলাকালীন critical section pre-empt হবে না। আসুন দেখি কিভাবে critical section সমস্যা সমাধান করতে interrupt ব্যবহার করা যায়। ধরা যাক, আমাদের দুটি প্রসেস, প্রসেস 1 এবং প্রসেস 2 আছে এবং তাদের এই লাল দিয়ে দেখানো critical section রয়েছে। এই critical section এ উভয় প্রসেস access করে এবং একই শেয়ার করা ডেটা পরিবর্তন করে। Critical section সমস্যার সমাধান এবং উভয় প্রসেস একই সময়ে critical section এ যাতে না থাকে সেটি নিশ্চিত করার জন্য interrupt disable করা হয়। অন্যদিকে, critical section প্রস্থানের সময়, interrupt আবার enable করা হয়। আমরা দেখেছি যে interrupt disable এর ফলে প্রসেসটির time slice শেষ করার পরেও প্রসেসটিকে pre-empt করা হবে না। মূলত, একটি timer interrupt হবে না যা প্রসেস 1 কে pre-empt করার থেকে বিরত করে। অতএব, আমরা দেখতে পাচ্ছি এটি একটি খুব সহজ সমাধান এবং প্রক্রিয়া 1 কোনও interrupt না থাকার ফলে execution চালিয়ে যেতে পারে। অনুরূপভাবে, যখন প্রসেস 2 এই critical section এ প্রবেশ করে, এটি interrupt disable করে; এবং critical section প্রস্থান করার সময় এটি পুনরায় interrupt enable করে। সুতরাং, এই interrupt disable এবং enable এর ব্যাপারটি critical section এ প্রবেশ করার আগে লকিং ব্যবস্থার অনুরূপ এবং critical section প্রস্থানকালে আনলকিং প্রক্রিয়ার অনুরূপ। লকিং নিশ্চিত করবে যে শুধুমাত্র একটি প্রসেস একটি নির্দিষ্ট সময়ে critical section এ প্রবেশ বা execute করে। যখন interrupt enable করা হয় অর্থাৎ আনলকিং করা হয় তখন অন্য প্রসেসকে critical section এ প্রবেশ বা execution এর সুযোগ প্রদান করা হয়। সুতরাং, critical section সমস্যার সমাধানের এই পদ্ধতিটি সহজ। এখানে interrupt disable বা enable করার জন্য কেবলমাত্র একটি instruction প্রয়োজন। এইভাবে, context switch প্রতিরোধ করা হয়। তবে, ইন্টারাপ্ট ব্যবহার করার সীমাবদ্ধতা হল এটির জন্য একটি উচ্চতর বিশেষাধিকার স্তর (higher privilege level) প্রয়োজন। সুতরাং, user space এ রান করা স্বাভাবিক অ্যাপ্লিকেশন প্রোগ্রামগুলি এইভাবে interrupt enable বা disable করতে সক্ষম হবে না। অতএব, এই সমাধান কেবলমাত্র কার্নেলে সঞ্চারিত কোডের জন্য প্রযোজ্য। সুতরাং এটি শুধুমাত্র কার্নেল বা অপারেটিং সিস্টেম interrupt enable বা disable করার অনুমতি দেয়। অতএব, এই সমাধান শুধুমাত্র অপারেটিং সিস্টেম কোডের জন্য প্রযোজ্য। আমরা আগেই দেখেছি যে এই interrupt enable বা disable করার পদ্ধতি multicore সিস্টেমের জন্য উপযুক্ত নয়; কারণ যখন মাল্টিকোর সিস্টেমে interrupt disable করা হয়, তখন এর অর্থ হবে যে কেবলমাত্র সেই কোডের সাথে সংশ্লিষ্ট interrupt গুলি disable হবে। অন্য দিকে, ইন্টারাপ্ট এখনও চলছে এমন অন্যান্য কোডের উপর অনুমোদিত হয়। এইভাবে একটি মাল্টিকোর সিস্টেমে যখন নিষ্ক্রিয় ইন্টারাপ্টগুলি চালু করা হয় তখন সমস্ত ইন্টারাপ্টগুলি কেবল সেই কোড বা প্রসেসরের কোডটি নিষ্ক্রিয় করা হয়। অন্য চালু প্রসেসগুলির interrupt এখনও হতে পারে। অন্য প্রসেসর যেমন যে প্রসেসটি প্রসেস 2 কে execute করছে interrupt disable দ্বারা প্রভাবিত হয় না। অতএব, এই সমাধান multicore সিস্টেমের জন্য উপযুক্ত নয়।

সুতরাং, আমরা পরবর্তীতে critical section সমস্যার জন্য আমাদের নিজের সমাধান তৈরি করার চেষ্টা করব। সুতরাং, আমরা একটি খুব সহজ সমাধান দিয়ে শুরু করবো, আমরা এর দুর্বলতাগুলি দেখে আমরা ধীরে ধীরে এই বিশেষ সমাধানটি সংশোধন করবো যতক্ষণ না আমরা এমন একটি বিন্দুতে পৌঁছাব যেখানে critical section সমস্যার সমাধান পাওয়া যায়। আসুন আমরা এটি দেখি।

(স্লাইড সময় 05:47 দেখুন)

সুতরাং আমাদের প্রথম প্রচেষ্টা দিয়ে শুরু করা যাক। সুতরাং মনে করি আমাদের প্রসেস 1 এ এই critical section এবং প্রসেস 2 তে এই critical section আছে। এবং দুটি critical section ই একই data manipulate করছে। এই shared data ছাড়াও, আমরা turn বলে অন্য একটি shared data সংজ্ঞায়িত করি এবং এর মান ধরি 1। তাই, আমরা আরো বিস্তারিতভাবে কী দেখি? যখন প্রসেস 1 এর execution শুরু হয় তখন এটি turn এর মান পায় 1, সুতরাং এই while loop অবিলম্বে exit হয় এবং critical section কার্যকর হয়। এখন যখন প্রসেস 1 critical section এ থাকে এবং প্রসেস 2 এই নির্দিষ্ট সময়ে এসে পৌঁছায়, তখন এটি turn এর মান 1 পায় এবং এই while loop এ ঘুরতে থাকে। এখান থেকে এটি প্রস্থান করতে সক্ষম হয় শুধুমাত্র যখন প্রথম প্রসেস 1 turn এর মান 2 করে, মনে রাখবেন যে turn একটি shared variable, তাই, প্রসেস 2 এর while loop টি শেষ হবে এবং প্রসেস 2 critical section এ প্রবেশ করবে। Critical section প্রস্থানকালে, প্রসেস 2 turn এর মান 1 করবে। turn এর মান 1 করার মানে হল যে প্রসেস 1 আবার critical section এ প্রবেশ করতে পারে। সুতরাং, মূলত যখন আমরা এই critical section সমস্যার সমাধান করার জন্য লকিং এবং আনলকিং প্রক্রিয়াটি দেখি, তখন আমরা দেখি যে এখানে উপস্থিত while loop টি লকিং প্রক্রিয়ায়, যেখানে আনলকিং প্রক্রিয়াটি হল তৃতীয় লাইনে যা turn এর মান 2 করে। আমরা যে দ্বিতীয় পর্যবেক্ষণটি করেছি তা হল এই অ্যালগরিদম বা critical section সমস্যার জন্য এই সমাধানটির প্রয়োজন যে প্রসেস 1 যখন critical section এ এক্সিকিউট হয় তখন turn এর মান 1 এবং তারপরে প্রসেস 1 প্রস্থান করার পরে turn এর মান 2 এ সেট করা হয়, প্রসেস 2 তখন এক্সিকিউট করবে। সুতরাং, একবার প্রসেস 2 তার এক্সিকিউশন কার্যকর করে, তারপর প্রসেস 1 চালানো হবে এবং এভাবে চলতে থাকে। তাই মূলত critical section এর জন্য প্রসেস 1 এবং প্রসেস 2 এর একটি পালা অনুযায়ী চলার আচরণ থাকবে। প্রথমে, প্রসেস 1 critical section চলবে কারণ turn এর মান 1। তারপর যখন এটি সমাপ্ত হয় তখন সেটি turn কে 2 তে পরিণত করে, তাই প্রসেস 2 চালানো হবে। এবং প্রসেস 2 তার critical section এর শেষে turn কে 1 মানে সেট করবে যার ফলে প্রসেস 1 critical section এ কার্যকর হবে। তাই মূলত critical section এর জন্য প্রসেস 1 এবং প্রসেস 2 এর একটি পালা অনুযায়ী চলার আচরণ থাকবে। বেশ স্বাভাবিকভাবেই কারন উভয় প্রসেসের মধ্যে turn এর মান share করা হয় এবং আমরা দেখেছি যে শুধুমাত্র একটি প্রসেস এই while এর অবস্থাটি সম্পন্ন করতে পারে, এবং while loop থেকে বেরিয়ে আসতে পারে। ঠিক একটি প্রসেস

একটি নির্দিষ্ট সময়ে critical section এ প্রবেশ করবে। অতএব, এই সমাধান mutual exclusion মান্য করে। এই সমাধানের দুটি সীমাবদ্ধতা আছে; প্রথমত busy wait। সুতরাং যখন প্রসেস 1 এই critical section সম্পাদন করে প্রসেস 2 এই while লুপের মধ্যে অবিচ্ছিন্নভাবে চলবে। লক্ষ্য করুন যে প্রসেস 2 ready অবস্থায় রয়েছে বা running অবস্থায় রয়েছে এটি blocked অবস্থায় নেই এবং সেইজন্য, এটি শক্তি ও সময় খরচ করবে, কারণ এটি নিয়মিতভাবে CPU তে চালানোর প্রয়োজন যাতে ক্রমাগত turn এর মানটি চেক করা যায়। অন্য সীমাবদ্ধতাটি হল critical section সমস্যার জন্য এই সমাধান প্রসেস 1 এবং প্রসেস 2 critical section এ অ্যাক্সেস alternate ভাবে হয়। এর মানে হল যে প্রথম প্রসেস 1 চলবে তারপর প্রসেস 2, তারপর প্রসেস 1, প্রসেস 2 এবং এ ভাবে চলবে। সুতরাং, এটি একটি সমস্যা তৈরি করে, বিশেষ করে critical section সমাধানগুলির progress এর প্রয়োজন। তাই, progress প্রয়োজনীয় শর্তটিতে উল্লেখ করা হয়েছে যে যদি কোনও প্রসেস critical section এ না থাকে এবং critical section এর জন্য একটি প্রসেস অনুরোধ করে, তাহলে এটিকে লক দেওয়া উচিত, যাতে এটি critical section এ চলতে পারে। আমরা দেখছি যে এই সমাধানটি progress এর প্রয়োজনীয়তা মান্য করবে না। উদাহরণস্বরূপ, ধরা যাক যে প্রসেস 2 প্রসেস 1 এর আগে চালানো শুরু করে। সুতরাং প্রসেস 2 এখানে আসে এবং এটি turn এর মান 1 থাকলে এটি চলতে থাকে যদিও প্রসেস 1 এখনো শুরু করেনি। তাই progress শর্ত পূরণ করার জন্য প্রসেস 2 আবার প্রয়োজনে critical section এ প্রবেশ করা উচিত। কিন্তু এটি হয়না, তাই এটি critical section সমস্যার সমাধানের জন্য একটি ভাল সমাধান নয়।

(স্লাইড সময় 11:16 দেখুন)

Critical section সমস্যা সমাধানের জন্য আমাদের প্রথম প্রচেষ্টা নিয়ে প্রধান ত্রুটি বা প্রধান সমস্যা এটি ছিল যে উভয় প্রসেস একটি সাধারণ turn flag ব্যবহার করেছে। এই turn flag প্রসেস 1 এ 2 সেট করা হয়েছিল এবং প্রসেস 2 এ 1 সেট করা হয়েছে; এবং এর ফলে দুটি প্রসেস critical section এ পর্যায়ক্রমে কার্যকর হতে বাধ্য হয়েছে। সুতরাং, আমরা আমাদের দ্বিতীয় প্রচেষ্টায় আরও ভাল করতে পারি যদি একটি একক shared flag এর বদলে দুটি প্রসেসের জন্য দুটি flag রাখি।

(স্লাইড সময় 12:02 দেখুন)

সুতরাং আসুন আমরা critical section সমস্যা সমাধান করার দ্বিতীয় প্রচেষ্টাটি দেখি। অতএব, আগের প্রচেষ্টার বিপরীতে আমরা এখন দুটি flag উপস্থাপন করছি। এই দুই flag এছাড়াও দুটি প্রসেসের মধ্যে share করা হয়; এর মানে হল যে এই flag p2_inside উভয় প্রসেস 1 এবং প্রসেস 2 থেকে অ্যাক্সেস করা যেতে পারে। তবে, এটিকে শুধুমাত্র প্রসেস 2 পরিবর্তন বা সংশোধন করে, তাই প্রসেস 2 flag টিকে false থেকে true বা true থেকে false করতে পারে, প্রসেস 1 শুধুমাত্র এই flag অবস্থাটি পড়ে; অর্থাৎ এটি শুধুমাত্র নির্ধারণ করে যে flag টি true না false। একইভাবে, দ্বিতীয় flag টি হল p1_inside। এখন এই p2_inside এবং p1_inside critical section এর

লকিং এবং আনলকিং এর সময় ব্যবহার করা হয়। মূলত প্রসেস 1 critical section এ প্রবেশের আগে দুটি কাজ করে, প্রথমত, এটি চেক করে যে দ্বিতীয় প্রসেস p2 critical section এ নেই এবং এটি তখনই হয় যখন p2_inside টি false হয়। সুতরাং, যদি p2_inside true হয়, তাহলে এর মানে হল যে দ্বিতীয় প্রসেসটি critical section এ কার্যকর করা হচ্ছে। তারপর এটি নিজের flag p1_inside কে true সেট করে এবং critical section এ চলে, যখন এটি critical section থেকে প্রস্থান করে এটি p1_inside কে false করে দেয়। একইভাবে, প্রসেস 2টি প্রথমে এই while loop এ চালানো হবে যতক্ষণ p1_inside টি false না হয় মানে প্রসেস 1 critical section থেকে বেরিয়ে এসেছে এবং critical section এ আর নেই। এবং এই ধরনের শর্তটি পাওয়ার পর, যখন while লুপটি সম্পন্ন হয় এবং এখানে execution হয়, যেখানে প্রসেস 2 p2_inside কে true সেট করে যা নির্দেশ করে যে এটি critical section এর ভিতরে এবং critical section এর শেষে এটি প্রস্থান করার পূর্বে p2_inside কে false সেট করে। এখন এই দ্বিতীয় সমাধানটিতে দুটি প্রসেসের alternate execution এর প্রয়োজন হয় না। উদাহরণস্বরূপ, যেহেতু p1_inside এবং p2_inside এর প্রাথমিক মানগুলি false, ধরি প্রসেস 2 প্রথমে execute করে, তাই এটা p1_inside এর মান false পায়, তাই এটি এই লুপ থেকে বেরিয়ে critical section এ প্রবেশ করে। অনুরূপভাবে, যদি প্রসেস 1 নির্বাহ করা শুরু না করে, তবে প্রসেস 2 প্রসেস 1 execution ছাড়াই critical section এ প্রবেশ করবে। যাইহোক, এই বিশেষ সমাধানের সমস্যা হল যে এটা mutual exclusion গ্যারান্টি করে না, প্রক্রিয়া 1, প্রক্রিয়া 2 একইসাথে critical section এ উপস্থিত হওয়া সম্ভব।

(স্লাইড সময় 15:45 দেখুন)

সুতরাং আসুন আমরা দেখি কিভাবে এই mutual exclusion আমাদের দ্বিতীয় প্রচেষ্টার জন্য ঠিক নয়। এখানে এই বিশেষ সারণী দেখায় যে CPU-র মধ্যে বিভিন্ন বিবৃতিগুলির সময়সীমার সাথে execution কার্যকর করা হয়, যেটি যখন এই while লুপটি কার্যকর করে এবং যখন এটি নীল রঙে থাকে, তখন এর মানে হল যে এটি প্রসেস 1 এর সাথে সম্পর্কযুক্ত। তারপর যখন p1_inside true এবং p2_inside true, তাই এটি সবুজ রঙে থাকে, সুতরাং এটি প্রসেস 2 এর সাথে সম্পর্কযুক্ত। এবং p1_inside টি true করে আবার প্রসেস 1 এর অনুরূপ। সুতরাং, এইভাবে CPU কিভাবে চালানো হবে নির্দেশাবলী দুটি প্রসেসের সাথে সংশ্লিষ্ট এবং আমাদের বেশ কিছু context switch মত রয়েছে যা execution এর সময় ঘটতে থাকে এবং এটি বিভিন্ন flag এর অবস্থা যা এক্সিকিউশনের অগ্রগতির হিসাবে। তাই আসুন শুরু করি প্রসেস 1 দিয়ে, প্রথমে আমরা সেট করেছি যে p1_inside এবং p2_inside দুটিই false যা ইঙ্গিত করে যে উভয় প্রসেসই critical section এ নেই। অতএব, এই বিশেষ while loop সম্পন্ন হবে, মূলত কারণ p2_inside false; এখন অনুমান করি একটি context switch আছে যা প্রসেস p2 সঞ্চালনের ফলে হয়, তাই এটি এই বিবৃতি চালানো হবে যখন p1_inside false আর তখন while loop ভাঙা হবে। এবং তারপর এটি p2_inside true সেট করে। এখন setup করার ঠিক পরেই এই ধরি context switch

তৈরি হয় এবং প্রসেস p1 টি চালানো অব্যাহত থাকবে এবং এটি p1_inside true হবে। মূলত context switch কারণে, এটি যেটি চালানো বন্ধ করা হয়েছে তা থেকে এটি চালানো অব্যাহত থাকবে যখন এটি while loop সম্পূর্ণ হওয়ার পরেই বন্ধ হয়ে যায় এবং তারপর এটি পরবর্তী নির্দেশনা দিয়ে চলে যে এখানে p1_inside true এর সমান। সুতরাং, এখানে আমরা দেখতে পাচ্ছি আমাদের একটি state আছে যেখানে p2_inside true নির্দেশ করে যে p2 critical section এ পাশাপাশি p1_inside true বলে নির্দেশ করে যে p1 ও critical section এ রয়েছে। অতএব, আমরা mutual exclusion অর্জন করতে পারিনি।

(স্লাইড সময় 18:15 দেখুন)

সুতরাং, critical section সমস্যা সমাধান করার জন্য এই দ্বিতীয় প্রচেষ্টার প্রধান সমস্যা ছিল যে আমরা ভিতরে দুটি flag p1_inside এবং p2_inside ব্যবহার করেছিলাম। যাইহোক, যখন আমরা এই while loop থেকে বিরতির পরে সেট করা হয়, যখন শুধুমাত্র এই নির্দিষ্ট সময় while loop টি বাস্তবায়ন হয় এবং এটি থেকে বিরত হয় শুধুমাত্র তারপর আমরা p1_inside true এবং p2_inside true সেট করি। মূলত, এর মানে কি এই যে p1_inside টি এবং p2_inside শুধুমাত্র critical section এ true সেট করা আছে, এবং এই সমস্যাটি তৈরি করেছে। সুতরাং আসুন আমরা দেখতে পারি যে আমরা আসলে এই বিন্যাস পরিবর্তন করতে পারি।

(স্লাইড সময় 19:03 দেখুন)

সুতরাং, আমরা তৃতীয় প্রচেষ্টা দিয়ে শুরু করবো যা critical section সমাধানের তৃতীয় প্রচেষ্টা। এবং আমরা কি করব কেবল একটি সহজ পরিবর্তন, তাই আমরা এই দুই flag হবে p1_inside পরিবর্তে, এখন p1 critical section প্রবেশ করতে চায়, এবং p2 critical section প্রবেশ করতে চায়। মূলত, এই flag গুলি প্রতিটি true সেট করা হয় যখন প্রাসঙ্গিক প্রসেসটি critical section এ প্রবেশ করতে চায়। সুতরাং, দ্বিতীয় প্রচেষ্টাটি নিয়ে আমরা এখানে ছোটখাট পরিবর্তন করি, যে প্রসেস 1 টি প্রথম p1_wantstoenter টি true দ্বারা সেট করে critical section এ প্রবেশ করার ইচ্ছা প্রকাশ করে। এবং শুধুমাত্র তারপর এটি নির্ধারণ করার চেষ্টা করবে, প্রক্রিয়া p2 critical section এ কী না। সুতরাং, মূলত এটি p1_wantstoenter কে true সেট করে প্রবেশ করতে চায় এবং তারপর এটি while loop এর মধ্যে সঞ্চালিত হবে যতক্ষণ না p2_wantstoenter false হয়। সুতরাং, যখন p2_wantstoenter false হয়, দ্বিতীয় প্রক্রিয়াটি critical section এ কার্যকর করা হয়ে গেছে, তারপর প্রসেস 1 critical section এ প্রবেশ করবে। শেষ পর্যায়ে, critical section টি সম্পূর্ণ সম্পন্ন হওয়ার পর p1 ক্লাগ p1_wantstoenter কে false এ সেট করে, false এর এই সেটিংটি প্রসেস 2 কে critical section এ প্রবেশ করার অনুমতি দেবে। লক্ষ্য করুন যে, এটির একমাত্র পার্থক্য হল, আমাদের আগের প্রচেষ্টার অনুরূপ ছিল যে আমরা ক্রিটিক্যাল সেকশনের ভেতর থেকে flag টিকে সরিয়ে while loop এর পরে আর while loop এর আগে নিয়েছি। তাই, আমরা দেখি যে এই বিশেষ সমাধান mutual exclusion অর্জন করে, অতএব, আগের প্রচেষ্টার বিপরীতে

যেখানে আমরা একটি শর্ত অর্জন করেছি যেখানে mutual exclusion না হয় এবং একই সময়ে আমরা গুরুত্বপূর্ণ বিভাগে দুইটি প্রসেস করি। এই সমাধানে, mutual exclusion করা হয়; মূলত, এটি নিশ্চিত করা হয় যে প্রক্রিয়া 1 যখন critical section এ থাকে, তখন প্রক্রিয়া 2 critical section এ নেই এবং তদ্বিপরীত। সুতরাং, progress গ্যারান্টি হয় না। মূলত, আমরা একটি state পেতে পারি যেখানে উভয় প্রসেস অবিরাম অপেক্ষাকৃত বিরামহীনভাবে এই লুপ সময় নির্বাহ করে এবং আমরা দেখেছি কিভাবে এটা ঘটে, তাই এই ধরনের একটি state একটি deadlocked state হিসাবে পরিচিত হয়।

(স্লাইড সময় 21:54 দেখুন)

সুতরাং আসুন আমরা এই বিশেষ ক্ষেত্রে deadlocked situation দেখতে পারি। তাই আসুন আমরা এই টেবিলে দেখি, যা দেখায় যে CPU কীভাবে সময়ের সাথে সাথে instructions চালায়, এবং এই টেবিলে বিদ্যমান flag গুলি বিভিন্ন মান দেখায়। সুতরাং আসুন শুরু করি প্রসেস 1 নির্বাহনের মাধ্যমে এবং CPU এই নির্দিষ্ট বিবৃতিটি চালায় এবং এটি p1 এর মান true এ সেট করতে চায়। তারপর একটি context switch আছে, যা প্রসেস 2 সঞ্চালন করে যা flag p2 কে true সেট করে। এখন আপনি দেখতে পাচ্ছেন যে উভয় প্রসেসই তাদের নিজস্ব flag true হিসাবে সেট করেছে। এখন যখন উভয় প্রসেস while লুপের প্রবেশ করে, আমরা দেখতে পাই যে উভয় ক্ষেত্রে প্রসেস 1 এবং সেইসাথে প্রসেস 2, এই flag টি true হিসাবে সেট করা আছে যে ইঙ্গিত দেয় যে উভয় প্রসেস বিরামহীনভাবে চলতে থাকবে while loop এ। মূলত, প্রসেস 1 প্রসেস 2 এর জন্য অপেক্ষা করছে তার flag কে false হিসাবে সেট করার জন্য, আবার প্রসেস 2 অপেক্ষা করছে বা প্রসেস 1 এর জন্য অপেক্ষা করছে যাতে তার flag টিকে false হিসাবে সেট করে। তাই, মূলত আমরা এমন একটি state এ পৌঁছেছি যেখানে প্রতিটি প্রসেস অপর প্রসেসের জন্য কিছু করার অপেক্ষায় আছে। সুতরাং, এটি একটি deadlocked অবস্থা হিসাবে পরিচিত হয় এবং এটা অপারেটিং সিস্টেমের সমস্যার মধ্যে যথেষ্ট হতে পারে।

(স্লাইড সময় 23:39 দেখুন)

সুতরাং, এই ছবিটি একটি deadlock এর একটি আরো স্পষ্ট দৃষ্টিভঙ্গি দিতে পারে, যেখানে আমাদের এখানে প্রসেস 1 এবং প্রসেস 2 এখানে উপস্থিত। এখন প্রসেস 1 while লুপের মধ্যে রয়েছে এবং এটি ক্রমাগত প্রসেস 2 এর জন্য অপেক্ষা করছে এই flag p2 কে false এ সেট করতে চায়। একইভাবে, প্রসেস 2টি যখন while লুপে ক্রমাগতভাবে অপেক্ষা করছে তখন এই flag p1 কে false এ সেট করতে অপেক্ষা করছে। যাইহোক, এখানে সমস্যা হল প্রক্রিয়া 1 এর এই flag এর মান পরিবর্তন প্রয়োজন কারণ এই flag শুধুমাত্র প্রক্রিয়া 1 দ্বারা পরিবর্তিত হতে পারে; অনুরূপভাবে প্রক্রিয়া 2 এর এই flag এর মান পরিবর্তন করতে পারে কারণ এই flag টি কেবলমাত্র p2 পরিবর্তন

করে। এবং তাই, আমরা দেখতে পাই যে একটি tie আছে; প্রক্রিয়া 1 তার flag এর মান পরিবর্তন করার জন্য প্রক্রিয়া 2 এর জন্য অপেক্ষা করছে; যাইহোক, প্রক্রিয়া 2 এর flag এর মান পরিবর্তন করতে পারছে না কারণ এটি flag এর মান পরিবর্তন করতে প্রক্রিয়া 1 এর জন্য অপেক্ষা করছে। সুতরাং, আমরা এখানে একটি চক্র পাই। সুতরাং, এই চক্রটি উভয় প্রক্রিয়া 1 এবং প্রক্রিয়া 2 একটি অবিরাম সময়ের জন্য একে অপরের জন্য অপেক্ষা করবে, তাই এটি একটি deadlock হিসাবে পরিচিত হয়। সুতরাং, পরে ভিডিওতে, আমরা কিভাবে deadlock গুলি সৃষ্টি করা হয় এবং কীভাবে সেগুলি আটকানো এবং এড়িয়ে যাওয়া যায় সে বিষয়ে বিশদ বিবরণ দেখব।

(স্লাইড সময় 25:18 দেখুন)

তাই আসুন আমরা দেখি যে তৃতীয় প্রচেষ্টাটির সাথে কি সমস্যা ছিল। মূলত আমরা প্রসেসটি ও তার flag সেট করেছিলাম এবং অন্যান্য প্রসেসের flag false হিসাবে সেট করার জন্য অপেক্ষা করছিলাম। এবং আমরা দেখেছি যে, যেহেতু উভয় প্রসেসই while লুপের মধ্যে ঢুকতে পারে এবং একে অপরের জন্য অপেক্ষা করতে পারে, আমাদের একটি deadlock অবস্থা থাকবে। এখন আমাদের পরবর্তী সর্বোত্তম জিনিস হচ্ছে যখন আমরা এই ধরনের একটি deadlock পরিস্থিতির সম্মুখীন হই, তখন আমাদের আসলেই deadlock পরিস্থিতি থেকে বেরিয়ে আসার একটি উপায় খুঁজতে হবে। মূলত আমরা এই দুটি প্রসেসের একটি এবং ঠিক একটি যাতে হোয়াইল লুপ থেকে প্রস্থান করে এবং critical section এ প্রবেশ করে তা নিশ্চিত করতে হবে।

(স্লাইড সময় 26:03 দেখুন)

সুতরাং, এই সমাধানটি পিটারসন তৈরি করেছেন এবং এটি পিটারসনের সমাধান হিসাবে পরিচিত, এবং মূলত এটি অন্য একটি variable যার নাম favored হিসাবে পরিচিত, তাই এটি একটি global variable যা দুটি প্রসেসের মধ্যে share করা হয় মূলতঃ, favored variable টি যে যখনই deadlock পরিস্থিতি ঘটে তখন এটি একটি প্রসেসের পক্ষ সমর্থন করে এবং সেই প্রক্রিয়াটি critical section এ প্রবেশ করে। যেখানে অন্য প্রক্রিয়াটি ততক্ষণ অপেক্ষা করতে থাকবে যতক্ষণ না পর্যন্ত দ্বিতীয় প্রক্রিয়াকে তার flag কে false হিসাবে সেট করবে। অতএব, মূলত পিটারসনের সমাধান অন্য প্রক্রিয়ায় favored এর মান নির্ধারণ করে যাতে এটি অন্য প্রক্রিয়ার পক্ষে critical section এ প্রবেশ করতে পারে। এখন loop exit এর জন্য এটি ব্যবহার করা হয়েছে যখন উভয় p1 এবং p2 critical section এ প্রবেশ করতে চাইছে অর্থাৎ যখন p1 এবং p 2 উভয়ই অপেক্ষা করছে তখন এই favored tie break করে। কারণ এই বিশেষ সমাধান কাজ করে যে, দুই প্রসেসের মধ্যে যে দুটি বিশেষত্ব আছে যে favored দুটি মান নিতে পারে, এবং সেইজন্য শুধুমাত্র একটি while লুপ সম্পূর্ণ হবে।

(স্লাইড সময় 27:31 দেখুন)

সুতরাং আসুন আমরা আরও বিস্তারিতভাবে এইটি দেখি। সুতরাং, আমরা দুটি প্রসেসগুলি p1 এবং p2 দেখেছি, এবং সবকিছু তৃতীয় প্রচেষ্টা হিসাবে আমরা চাই যে p1 true সেট করতে চায় এবং তারপর একটি while লুপ আছে এবং তারপর critical section এবং তারপর p1 চাই false সেট করতে। এখন পিটারসন সমাধান এ এখানে একটি favored যোগ করা হয়েছে এবং এই favored এখানে 2 সেট করা হয় এবং favored দ্বিতীয় প্রক্রিয়ায় 1 এ সেট করা হয়। সুতরাং, অন্য সব ক্ষেত্রে এই বিশেষ পিটারসন এর সমাধান আমাদের তৃতীয় প্রচেষ্টা হিসাবে critical section এর সমাধান তৈরি করতে সাহায্য করে। তবে একমাত্র পার্থক্যটি হল যখন একই সময়ে while লুপে উভয় প্রসেসই প্রবেশ করে। এই ক্ষেত্রে, favored মান নির্ধারণ করবে দুটি প্রক্রিয়ার মধ্যে কোনটি critical section এ প্রবেশ করে। সুতরাং আসুন আমরা বলি যে প্রক্রিয়া 2 হল favored এবং এটি critical section এ প্রবেশ করে, এবং এর critical শেষের দিকে এটি p2 কে false সেট করে। সুতরাং, মনে রাখবেন যে এই পুরো সময় p1 এখনও অপেক্ষা করে বা এখনও while লুপ মধ্যে নির্বাহ করা হয়। এখন যখন প্রক্রিয়া 2 তার flag false হিসাবে সেট করে দেয়, তা তখনই while লুপ ভেঙে যায়, ফলে p1 critical section এ প্রবেশ করবে। সুতরাং, কেবলমাত্র যখন প্রক্রিয়া 2 critical section থেকে প্রস্থান করে তখন p1 critical section প্রবেশ করবে, এইভাবে mutual exclusion অর্জন করে। এছাড়াও, আমরা দেখেছি যে bounded wait সমস্যা সমাধান করা হয় এবং পাশাপাশি progress টিও সমাধান করা হয়। পিটারসন এর সমাধান দুটি প্রক্রিয়ার জন্য খুব দক্ষতার সঙ্গে কাজ করে। পরবর্তী ভিডিওতে, আমরা বেকারি অ্যালগরিদম দেখতে পাব, যা সিস্টেমের একাধিক প্রসেসের ক্ষেত্রে critical section এর জটিল সমস্যার সমাধান করতে ব্যবহৃত হয়। সুতরাং, বেকারি অ্যালগরিদম critical section এর সমস্যা সমাধান করে যখন প্রসেসের সংখ্যা দুই এর বেশি হয়। ধন্যবাদ।